

Docker compose local example

Systems Integration

__/__/2025

The application

- A Python Flask sample application with a Redis in-memory data store.
- A user, using a browser, can add a student name to the database by mean of a POST HTTP request.
- A user, using a browser, can list all the students in the database by mean of a GET HTTP request.
- Two containers:
 - the Flask based web server (app) which wait on the internal 5000 TCP port.
 - the Redis in-memory database (redis).
- The app and redis containers share a network. **Containers on this network can refer each other by service name.**
- The app container only uses another network and communicates with the users.
- The app container exposes the 5000 TCP port to the 8080 TCP port of the host.

Files

vedi Esempio uso Docker Compose su singolo host: ComposeExample1

- requirements.txt
- app.py
- Dockerfile
- docker-compose.yml

- README.txt

The app python application

cd ~

mkdir ComposeExample1

cd ComposeExample1

Edit the **app.py** file

```
from flask import Flask, request, jsonify
from redis import Redis
```

```
app = Flask(__name__)
redis = Redis(host="redis", db=0, socket_timeout=5, charset="utf-8",
              decode_responses=True)
```

```
@app.route('/', methods=['POST', 'GET'])
def index():
```

```
    if request.method == 'POST':
        name = request.json['name']
        redis.rpush('students', {'name': name})
        return jsonify({'name': name})
```

```
    if request.method == 'GET':
        return jsonify(redis.lrange('students', 0, -1))
```

USE container
name as host
name for DNS
resolution

ADD a student

LIST student 4

Specify the dependencies

- Our application has two dependencies, Flask and Redis, which you can see in the first two lines. These dependencies must be defined before we can execute the application.

Create the file requirements.txt

flask

redis<3.0.0

Docker Compose vs. Docker Swarm

- Docker Compose is a tool to manage multi-container applications. **Docker Compose can manage multi-container applications on a single machine only. It does not work on computer clusters, accross multiple machines.**
- **Docker Swarm, another Docker product, is designed to manage multi-container application stacks on computer clusters. You can use Docker Swarm to deploy and run your stack accross multiple machines.**
- *Docker Compose and Docker Swarm can both use the same Compose file to deploy and run application stacks.*
- Docker Compose was released first and is used it to create applications with multiple services, connected them with custom networks and run them on a single machine.
- Later on, the cluster management features were added to Docker with Docker Swarm and the Compose file has been made compatible with the Swarm mode.
- ***This implies, that you'll find entries in the Compose file reference online that are compatible with either Docker Compose or Docker Swarm. Whenever some feature is limited to either Compose or Swarm, it is clearly highlighted in the documentation.***

The structure of the Compose file

- The Compose file (docker-compose.yml) contains several entries structured hierarchically. The top level entries are the following:
- **version** - specifies the version of the Compose file reference.
- **services** - specifies the services in your application, we used it to define the containers.
- **networks** - you can define the networking set-up of your application here.
- **volumes** - you can define the volumes used by your application here.
- **secrets** - secrets are related to Swarm mode only, you can use them to provide secret information like passwords in a secure way to your application services.
- **configs** - configs lets you add external configuration to your containers. Keeping configurations external will make your containers more generic. Configs is available both in Compose and in Swarm mode.
- Typically, the **services** entry contains more section for each service, in particular there are entries for build configuration and runtime configuration.
- The **deploy** section only applies in Swarm mode.

Service configuration options (1/2)

- Build configuration
 - You use the **build** section to specify the details of the build. You can define your build **context**, build arguments (**args**), dockerfile **location** and even the **target** build stage in a multi-stage build.
- Runtime configuration
 - You can specify runtime parameters in the Config file the same way you can specify them with the docker run command . The **image** section specifies the name of the image to be used to start up containers. If your service has a build section, the resulting image of the build will be tagged with the name that you specify in the image section. You can override the container command in the command section in the Compose file. You can specify an entry point with **entrypoint**
 - You can specify a custom **container_name**, this may be useful if you need the container name in a script, for example. Compose can scale your services, meaning start up **multiple replicas** of a service as you'll see later. If you use a custom container_name, you cannot scale that service beyond one container.
 - You can pass environment variables with **environment**.

Service configuration options (2/2)

- Deployment configuration
 - **The deploy section only applies in Swarm mode.** It includes deployment configuration and resource limitations, update and rollback preferences and more. **Version 3 of the Compose file reference does not include deployment config** for Compose based services.
 - Version 2 of the Compose file format has CPU, memory and other resource configuraiton options, you can use version 2 if you want to use Compose in production.

Docker Compose command and subcommands

- The Docker Compose command has various subcommands. For help : (1/2)
`docker-compose --help`
- Build images with Docker compose
 - The build subcommand will build all the images that are specified in the Compose file. The command will go through all services in the Compose file and build the ones that have a build section defined.
`docker-compose build`
 - Build images and create the stack. The command `docker-compose up` will only build your image if the image does not exist, but if it exists it will not re-build it every time you start up your application.
`docker-compose up`
 - The command `docker-compose up --build` will re-build your image if the Dockerfile or image files have changed.
`docker-compose up --build`
 - If you need to initiate a build with no cache you can use the `docker-compose build --no-cache` command.

Use build arguments (1/2)

- Each build section can specify multiple options on multiple lines. To achieve this we moved the build context path to the line context: . and we added the build argument under args. We can add multiple arguments if we need to.

```
build: .
```

```
build:  
context: .  
args:  
    - IMAGE_VERSION=3.7.0-alpine3.8  
      PIPPO=PAPPA
```

- Each argument defined in Compose file can be used in the Dockerfile., but we need to declare it before using it. This argument does not create an environment variable of a container.
- For example, in a Dockerfile:
ARG IMAGE_VERSION
FROM python:\$IMAGE_VERSION
- The ARG instruction denotes that we'll use the argument IMAGE_VERSION. Since we are using the argument in the FROM instruction we have to put ARG before FROM. Then we use the value of the build argument on the next line: FROM python:\$IMAGE_VERSION.

Use build arguments (2/2)

Dockerfile

ARG IMAGE_VERSION

FROM python:\$**IMAGE_VERSION**

WORKDIR /usr/src/app

COPY requirements.txt ./

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py ./

ENV FLASK_APP=app.py

CMD flask run --host=0.0.0.0

Manage multi-container applications

- The following command will list the containers in your application
`docker-compose ps`
- You can access the Docker native logs of all containers using the command
`docker-compose logs`
or for a give service
`docker-compose logs SERVICENAME`
- NB: The log file of a container can be seen with:
`docker inspect container | grep LogPath`
- To see the processes in your containers use the following command
`docker-compose top`
- You also have subcommands to start, stop, kill and restart containers for all or some servicessome in your multi-container application with the start, stop, kill and restart subcommands. In particular, the **down** subcommand stop and delete all the containers, networks and volumes created by docker-compose up.
`docker-compose down`
- The argument `--rmi all` delete the container images also.
`docker-compose down --rmi all`

Execute commands in a running container

- We can execute commands in running container:
- The following command will list the containers in your application
docker-compose **exec** service commandname arguments

Run one-off commands

- Sometimes you want to run one-off (una tantum) commands in a container. This can be done using the `docker-compose run` command
 - `docker-compose run service commandname arguments`
- **This is different from the `exec` subcommand**, because **`run`** is not executed against a running container, **`run` will start a new container to execute the command**. This is useful if you don't want to mess with the running containers.
- There is one key difference between `run` and `up`.
 - When you use **`run`**, Docker will not map the ports to the host machine. This behavior lets you run one-off commands in new containers even when your stack is running and your container that you started with `docker-compose up` is already mapped to the host port.
- If you need to map a port, you can use the `-p` flag of `docker-compose run`.
 - `docker-compose run -p service commandname arguments`

Docker Compose networks (1/2)

- You can connect your service with custom, user defined networks using the Compose file, as in the following example:

```
version: '3'

services:
  app:
    build:
      context: .
      args:
        - IMAGE_VERSION=3.7.0-alpine3.8
    image: takacsmark/flask-redis:1.0
    environment:
      - FLASK_ENV=development
    ports:
      - 8080:5000
    networks:
      - mynet
  redis:
    image: redis:4.0.11-alpine
    networks:
      - mynet

networks:
  mynet:
```

Docker Compose networks (2/2)

- We have defined a user defined network **under the top-level networks section** at the end of the file and called the network *mynet*.
- This is a bridge network, which means that it's a network on the host machine that is isolated from the rest of the host network stack.
- **Under each service we added the networks key to specify that these services should connect to mynet.**
- Let's create the application stack running the command:

```
docker-compose up -d
```


and inspect the configuration of the created network

```
composeexample1__mynet
```



```
docker network inspect composeexample1_mynet
```
- Compose has created the new network and the containers and attached them to this network. Note that the network name has been decided by Docker.
- You can use the top-level networks section to specify a more complex network configuration, too. You can specify multiple networks and attach containers to various network segments of your architecture.

Volumes and Compose (1/2)

- The top level volumes section lets you create named volumes for your application.
- Volumes are used to store data outside containers. This is handy if you destroy a container and create a new one, your data will not be lost with the container, but it'll be persisted on the host machine in a volume.
- Our example uses one volume currently that is used by the Redis image:
- We defined the mydata volume in the top-level volumes section at the end of the file and we mapped this volume with the volumes option to the redis service.

```
version: '3'
```

```
services:
```

```
redis:
```

```
  image: redis:4.0.11-alpine
```

```
  networks: - mynet
```

```
  volumes:
```

```
    - mydata:/data
```

```
volumes:
```

```
  mydata:
```

Volumes and Compose (2/2)

- The volume will be stored in a directory in `/var/lib/docker/volumes/`
- Use this command to see the available volumes
`docker volume ls`
- Use this command to see details about a given volume
`docker volume inspect nome_volume`
`docker volume inspect composeexample1_mydata`
- Use this command to remove all unused volumes
`docker volume prune`
- Use this command to remove a given volume
`docker volume rm nome_volume`
`docker volume rm composeexample1_mydata`

Use `env_file` to pass environment variables

- We usually prefer to use environment files in our projects to pass environment variables to our applications. Use the **`env_file`** option to add an environment file to your service's configuration to pass environment variables.
- The top level volumes section lets you create named volumes for your application.
- Volumes are used to store data outside containers. This is handy if you destroy a container and create a new one, your data will not be lost with the container, but it'll be persisted on the host machine in a volume.
- Our example uses one volume currently that is used by the Redis image:
- We defined the mydata volume in the top-level volumes section at the end of the file and we mapped this volume with the volumes option to the redis service.

```
version: '3'
```

```
services:
```

```
redis:
```

```
  image: redis:4.0.11-alpine
```

```
  env_file: env.txt
```

```
  networks: - mynet
```

```
  volumes:
```

```
    - mydata:/data
```

```
file env.txt
```

```
FLASK_ENV=developmentdio
```

Run the application

- Build command:
`docker-compose build`
- Run command:
`docker-compose up -d`
- Check ports:
`docker exec -it ComposeExample1_app_1 netstat -atn`
`docker exec -it ComposeExample1_redis_1 netstat -atn`
`docker container inspect composeexample1_app_1 --format='{{json .NetworkSettings.Ports}}'`
- Client Commands:
 Add a student:
 `curl --header "Content-Type: application/json" \ --request POST --data '{"name":"Rossi"}' localhost:8080`
 Lists students:
 `curl localhost:8080`
- Stop and remove containers, networks and volume created by "up":
`docker-compose down`
- Stop and remove containers, networks, volume and images created by "up":
`docker-compose down --rmi all`

Your first scaled service

- uff, per la prossima volta